COMPILER DESIGN LAB – PROGRAMS

Programming Environment: Ubuntu / Google Colab
Programming Languages: C, Lex (Flex), Yacc (Bison)

Experiment 1: Removing White Spaces

Problem Statement: Write a Lex program to eliminate white spaces between words in a given text.

Program: `space.l`

```
%{
#include<stdio.h>
%}
%%
[ \t\n]+ ;
. { printf("%s",yytext); }
%%
int main()
{
    yylex();
    return 0;
}
int yywrap()
{
    return 1;
}
```

Compilation:

```
flex space.l
gcc lex.yy.c -o space -lfl
./space
```

Sample Input:

```
Welcome to Compiler Design Lab
```

Output:

```
WelcometoCompilerDesignLab
```

Experiment 2: Removing Comments from a C Program

Problem Statement: Write a Lex program to remove both single-line (`//`) and multi-line (`/* */`) comments from a C program.

Program: `comment.l`

```
%{
#include<stdio.h>
%}
%%
"//".* ;
"/*"([^*]|\*+[^*/])*\*+"/" ;
.|\n { ECHO; }
%%
int main()
{
    yylex();
    return 0;
}
int yywrap()
{
    return 1;
}
```

Compilation:

```
flex comment.l
gcc lex.yy.c -o comment -lfl
./comment < input.c
```

Experiment 3: Lexical Analyzer for C Tokens

Problem Statement: Write a Lex program to identify different tokens in a subset of the C language, namely keywords, identifiers, operators, constants, and special characters.

Program: `tokens.l`

```
%{
#include<stdio.h>
%}
%%
"int"|"float"|"char"|"if"|"else"|"while"|"return"
    { printf("Keyword: %s\n",yytext); }

[a-zA-Z_][a-zA-Z0-9_]*
    { printf("Identifier: %s\n",yytext); }

[0-9]+(\.[0-9]+)?
    { printf("Constant: %s\n",yytext); }

"=="|"!="|"<="|">="|"++"|"--"|"+"|"-"|"*"|"/"|"="|"<"|">"
    { printf("Operator: %s\n",yytext); }

[(){};,]
    { printf("Special character: %s\n",yytext); }

[ \t\n]+ ;
.
    { printf("Unknown: %s\n",yytext); }
%%
int main()
{
    yylex();
    return 0;
}
int yywrap()
{
    return 1;
}
```

Compilation:

```
flex tokens.l
gcc lex.yy.c -o tokens -lfl
./tokens
```

Experiment 4: Recognizing Keywords Only

Problem Statement: Write a Lex program to identify and display only the keywords present in a C program.

Program: `keyword.l`

```
%{
#include<stdio.h>
%}
%%
"int"|"float"|"char"|"if"|"else"|"while"|"return"
    { printf("Keyword: %s\n",yytext); }

[ \t\n]+ ;
.
%%
int main()
{
    yylex();
    return 0;
}
int yywrap()
{
    return 1;
}
```

Compilation:

```
flex keyword.l
gcc lex.yy.c -o keyword -lfl
./keyword
```

Experiment 5: Recognizing Java Tokens

Problem Statement: Write a Lex program to recognize tokens such as keywords, identifiers, constants, operators, and special symbols in a given Java program.

Program: `java.l`

```
%{
#include<stdio.h>
%}
%%
"import"|"class"|"public"|"static"|"void"|"new"|"int"|"float"|"return"
    { printf("Keyword: %s\n",yytext); }

[a-zA-Z_][a-zA-Z0-9_]*
    { printf("Identifier: %s\n",yytext); }

[0-9]+
    { printf("Constant: %s\n",yytext); }

"=="|"="|"+"|"-"|"*"|"/"
    { printf("Operator: %s\n",yytext); }

[(){};,.\[\]]
    { printf("Special symbol: %s\n",yytext); }

[ \t\n\r]+ ;

\"[^\"\n]*\"
    { printf("String: %s\n",yytext); }


.
    { printf("Other: %s\n",yytext); }
%%
int main()
{
    yylex();
    return 0;
}
int yywrap()
{
    return 1;
}
```

Compilation:

```
flex java.l
gcc lex.yy.c -o java -lfl
./java < input.java
```

Experiment 6: Eliminating Direct Left Recursion

Problem Statement: Write a C program to eliminate direct left recursion from a given Context-Free Grammar (CFG).

Conversion Rule:

Original grammar:

```
A -> Aα | β
```

After eliminating direct left recursion:

```
A  -> βA'
A' -> αA' | ε
```

Program: `direct.c`

```
#include<stdio.h>
#include<string.h>

int main()
{
    char p[20][20],a[20][20],b[20][20];
    int n,i,x=0,y=0;

    printf("Number of productions: ");
    scanf("%d",&n);

    for(i=0;i<n;i++)
        scanf("%s",p[i]);

    for(i=0;i<n;i++)
    {
        if(p[i][0]=='A')
            strcpy(a[x++],p[i]+1);
        else
            strcpy(b[y++],p[i]);
    }

    if(x==0)
    {
        printf("No direct left recursion");
        return 0;
    }

    printf("A -> ");
    for(i=0;i<y;i++)
        printf("%sA' ",b[i]);

    printf("\nA' -> ");
    for(i=0;i<x;i++)
        printf("%sA' | ",a[i]);

    printf("epsilon\n");
    return 0;
}
```

Sample Input:

```
3
Aa
Ab
c
```

Output:

```
A -> cA'
A' -> aA' | bA' | epsilon
```

Experiment 7: Eliminating Indirect Left Recursion

Problem Statement: Write a program to eliminate indirect left recursion from a given Context-Free Grammar using forward substitution and direct left recursion elimination.

Example Grammar:

```
A -> Bb | a
B -> Ac | d
```

Step 1: Substitute A into B

```
B -> Bbc | ac | d
```

Step 2: Eliminate direct left recursion

```
B  -> acB' | dB'
B' -> bcB' | epsilon
```

Algorithm:

1. Arrange non-terminals in an order.
2. Substitute earlier non-terminals in the productions.
3. Convert indirect left recursion into direct left recursion.
4. Eliminate direct left recursion using the standard formula.

Note: We discussed this transformation and its algorithm, but have not yet completed a general executable C program for arbitrary grammars.

Experiment 8: Finding FIRST and FOLLOW Sets

Problem Statement: Write a C program to calculate FIRST and FOLLOW sets of a given Context-Free Grammar.

Definitions:

- FIRST: Set of terminals that can appear at the beginning of a string derived from a grammar symbol.
- FOLLOW: Set of terminals that can appear immediately after a non-terminal.
- # represents epsilon.
- $ represents the end marker.

FIRST Rules:

1. If X is a terminal, FIRST(X) = {X}.
2. If X derives epsilon, include epsilon in FIRST(X).
3. For a production X → YZ, add FIRST(Y) excluding epsilon.
4. If Y derives epsilon, consider FIRST(Z).

FOLLOW Rules:

1. Add $ to FOLLOW of the start symbol.
2. For A → αBβ, add FIRST(β) excluding epsilon to FOLLOW(B).
3. If β derives epsilon, add FOLLOW(A) to FOLLOW(B).
4. Repeat until no more symbols can be added.

Compilation:

```
gcc firstfollow.c -o ff
./ff
```

Note: The earlier compact FIRST/FOLLOW code we discussed was only an illustration and does not correctly handle all grammar cases. A reliable general implementation should use repeated set updates until no new symbols are added.

Experiment 9: Predictive Parsing Table and LL(1)

Problem Statement: Construct a non-recursive predictive parsing table using FIRST and FOLLOW sets and determine whether a grammar is LL(1).

Algorithm:

1. Calculate FIRST and FOLLOW sets.
2. For every production A → α, find FIRST(α).
3. Insert A → α in the table for each terminal in FIRST(α), excluding epsilon.
4. If FIRST(α) contains epsilon, insert A → α for every terminal in FOLLOW(A).
5. Check for multiple productions in the same table entry.
6. If no entry contains multiple productions, the grammar is LL(1).

Important: We covered the algorithm and theory, but have not yet written a complete executable predictive parser.

Common Compilation Commands

For Lex programs:

```
flex program.l
gcc lex.yy.c -o output -lfl
./output
```

For C programs:

```
gcc program.c -o output
./output
```

For Lex programs that read an input file:

```
./output < input.c
```

Experiment 10: Generate Machine Code from Three-Address Code

Problem Statement: Write a C program to generate target machine code (assembly instructions) from a given Three-Address Code (TAC).

Program: machinecode.c

```
#include<stdio.h>

#include<string.h>

void generate(char op,char *r,char *a,char b)

{

switch(op)

{

case '+':

printf("MOV R0,%s\nADD R0,%s\nMOV %s,R0\n",a,b,r);

break;

case '-':

printf("MOV R0,%s\nSUB R0,%s\nMOV %s,R0\n",a,b,r);

break;

case '':

printf("MOV R0,%s\nMUL R0,%s\nMOV %s,R0\n",a,b,r);

break;

case '/':

printf("MOV R0,%s\nDIV R0,%s\nMOV %s,R0\n",a,b,r);

break;

case '=':

printf("MOV R0,%s\nMOV %s,R0\n",a,r);

break;

default:

printf("UNKNOWN OPERATION\n");
```

}

}

int main()

{

char tac[][20]={

"t1 = a + b",

"t2 = c * d",

"t3 = t1 - t2",

"x = t3"

};

```
 int n=4,i;
 for(i=0;i<n;i++)
 {
     char r[10],a[10],b[10],op;
     int k=sscanf(tac[i],"%s = %s %c %s",r,a,&op,b);

     if(k==4)
         generate(op,r,a,b);
     else
     {
         sscanf(tac[i],"%s = %s",r,a);
         generate('=',r,a,"");
     }
     printf("\n");
 }
 return 0;
```

}

Compilation:

gcc machinecode.c -o machinecode

./machinecode

Expected Output (sample):

MOV R0,a

ADD R0,b

MOV t1,R0

MOV R0,c

MUL R0,d

MOV t2,R0

MOV R0,t1

SUB R0,t2

MOV t3,R0

MOV R0,t3

MOV x,R0

Experiment 11: Generate Three-Address Code

Problem Statement: Write a C program to generate Three-Address Code for a given arithmetic expression using an expression tree.

Program: tac.c

```
#include<stdio.h>

#include<stdlib.h>

#include<string.h>

int count=1;

typedef struct Node

{

char data[10];

struct Node *left,*right;

} Node;

Node* create(char *s)
```

{

Node *n=malloc(sizeof(Node));

strcpy(n->data,s);

n->left=n->right=NULL;

return n;

}

char* generate(Node *root)

{

if(root->left==NULL && root->right==NULL)

return root->data;

```
 char *a=generate(root->left);
 char *b=generate(root->right);

 char *t=malloc(10);
 sprintf(t,"t%d",count++);

 printf("%s = %s %s %s\n",t,a,root->data,b);
 return t;
```

}

int main()

{

Node *root=create("-");

root->left=create("+");

root->right=create("d");

```
 root->left->left=create("a");
 root->left->right=create("*");

 root->left->right->left=create("b");
 root->left->right->right=create("c");

 printf("Expression: a + b * c - d\n");
 printf("Three Address Code:\n");

 generate(root);
 return 0;

}
```

Compilation:

gcc tac.c -o tac

./tac

Expected Output:

Expression: a + b * c - d

Three Address Code:

t1 = b * c

t2 = a + t1

t3 = t2 - d

Experiment 12: Implement LALR Parsing

Problem Statement: Write a C program to implement LALR parsing and check whether a given input string is accepted by the grammar.

Note: The supplied program is a simplified stack-based parser, not a genuine LALR parser. It does not construct LALR item sets, merge LR(1) states, or use an ACTION/GOTO parsing table. For submission, the following is a simplified version of the supplied code.

Program: lalr.c

#include<stdio.h>

#include<string.h>

char stack[100];

int top=-1;

void push(char c)

{

stack[++top]=c;

}

int main()

{

char input[]="abab";

int i;

```
 for(i=0;input[i];i++)
 {
     if(input[i]=='a'||input[i]=='b')
         push(input[i]);
     else
     {
         printf("Invalid input\n");
         return 0;
     }
 }

 printf("Input: %s\n",input);
 printf("Input is accepted by the simplified parser.\n");

 return 0;
```

}

Compilation:

gcc lalr.c -o lalr

./lalr

Expected Output:

Input: abab

Input is accepted by the simplified parser.

Updated Experiment List

Experiment	Program
1	Removing White Spaces
2	Removing Comments
3	Lexical Analyzer for C Tokens
4	Recognizing Keywords
5	Recognizing Java Tokens
6	Eliminating Direct Left Recursion
7	Eliminating Indirect Left Recursion
8	Finding FIRST and FOLLOW Sets
9	Predictive Parsing Table and LL(1)
10	Generating Machine Code from Three-Address Code
11	Generating Three-Address Code
12	Implementing LALR Parsing

Important for submission: Experiments 10 and 11 are simplified executable versions of the supplied programs. Experiment 12 is only a demonstration parser, not an actual LALR implementation. If your lecturer specifically evaluates LALR parsing, use a proper LALR parser generator such as Bison rather than claiming this simplified code implements LALR.

can you give in same font as above

Sure! Here are Experiments 10, 11 and 12 in the same simple format as your earlier programs, with consistent code formatting and headings.

Experiment 10: Generate Machine Code from Three-Address Code

Problem Statement: Write a C program to generate machine code from a given Three-Address Code.

Program: machinecode.c

```
#include<stdio.h>
#include<string.h>
void generate(char op,char *r,char *a,char b)
{
switch(op)
{
case '+':
printf("MOV R0,%s\nADD R0,%s\nMOV %s,R0\n",a,b,r);
break;
case '-':
printf("MOV R0,%s\nSUB R0,%s\nMOV %s,R0\n",a,b,r);
break;
case '':
printf("MOV R0,%s\nMUL R0,%s\nMOV %s,R0\n",a,b,r);
break;
case '/':
printf("MOV R0,%s\nDIV R0,%s\nMOV %s,R0\n",a,b,r);
break;
case '=':
printf("MOV R0,%s\nMOV %s,R0\n",a,r);
break;
}
}
int main()
{
char tac[][20]={
```

"t1 = a + b",

"t2 = c * d",

"t3 = t1 - t2",

"x = t3"

};

int i;

```
 for(i=0;i<4;i++)
 {
     char r[10],a[10],b[10],op;
     int k=sscanf(tac[i],"%s = %s %c %s",r,a,&op,b);

     if(k==4)
         generate(op,r,a,b);
     else
     {
         sscanf(tac[i],"%s = %s",r,a);
         generate('=',r,a,"");
     }
     printf("\n");
 }
 return 0;
```

}

Compilation:

gcc machinecode.c -o machinecode

./machinecode

Experiment 11: Generate Three-Address Code

Problem Statement: Write a C program to generate Three-Address Code for a given arithmetic expression using an expression tree.

Program: tac.c

#include<stdio.h>

#include<stdlib.h>

#include<string.h>

```
int count=1;

typedef struct Node

{

char data[10];

struct Node *left,*right;

} Node;

Node* create(char *s)

{

Node *n=malloc(sizeof(Node));

strcpy(n->data,s);

n->left=n->right=NULL;

return n;

}

char* generate(Node *root)

{

if(root->left==NULL && root->right==NULL)

return root->data;

 char *a=generate(root->left);
 char *b=generate(root->right);

 char *t=malloc(10);
 sprintf(t,"t%d",count++);

 printf("%s = %s %s %s\n",t,a,root->data,b);
 return t;

}

int main()

{
```

Node *root=create("-");

root->left=create("+");

root->right=create("d");

```
 root->left->left=create("a");
 root->left->right=create("*");

 root->left->right->left=create("b");
 root->left->right->right=create("c");

 printf("Expression: a + b * c - d\n");
 printf("Three Address Code:\n");

 generate(root);
 return 0;
```

}

Compilation:

gcc tac.c -o tac

./tac

Experiment 12: Implement LALR Parsing

Problem Statement: Write a C program to implement LALR parsing and check whether a given input string is accepted by the grammar.

Program: lalr.c

#include<stdio.h>

#include<string.h>

char stack[100];

int top=-1;

void push(char c)

{

stack[++top]=c;

}

int main()

{

char input[]="abab";

int i;

```
 for(i=0;input[i];i++)
 {
     if(input[i]=='a'||input[i]=='b')
         push(input[i]);
     else
     {
         printf("Invalid input\n");
         return 0;
     }
 }

 printf("Input: %s\n",input);
 printf("Input is accepted by the simplified parser.\n");

 return 0;
```

}

Compilation:

gcc lalr.c -o lalr

./lalr